

Attitude Estimation: A Comparison of Filters

Akhil Sankar

Abstract

This report presents a first-principles design and implementation of a real-time Attitude Heading Reference System (AHRS), representing the state estimator for a self-stabilizing drone flight control loop. The system implements a Madgwick orientation filter, fusing 3-axis gyroscope and accelerometer sensor modalities to estimate 3D orientation. Quaternion representation is used to avoid gimbal lock singularities. A gradient-descent based optimization is used to correct for gyroscope drift using gravity vector observations. The filter is executed on a bare-metal embedded platform using an underlying serial communication bus.

1. Introduction

Accurate measurement of 3D orientation is fundamental to a wide range of applications including aerospace navigation. Traditional estimation methods such as optical tracking or GPS-based solutions are limited by environmental constraints, requiring line-of-sight or outdoor operation. Inertial Measurement Units (IMUs) offer a self-contained alternative, allowing for estimation without external references.

An IMU typically combines a 3-axis gyroscope, accelerometer, and optionally magnetometer to measure rotational velocity, linear acceleration, and local magnetic field respectively. Gyroscopes can measure instantaneous angular rates; however, performing numerical integration to obtain orientation accumulates drift error over time due to sensor noise and bias. Accelerometers and magnetometers provide absolute orientation references relative to the Earth's gravitational and magnetic fields, but are susceptible to motion-induced disturbances and magnetic interference.

A filtering algorithm, such as the Madgwick Filter or Extended Kalman Filter can be used to fuse telemetry from the different sensor modalities, guaranteeing that error monotonically and asymptotically tends to zero. An Attitude Reference Heading System (AHRS) is a self-contained embedded system that consists of an IMU and such a filtering algorithm. This paper presents a bare-bones design of an AHRS, focusing on connecting filter mathematics with their practical implementation on a bare-metal ARM microcontroller.

This paper is therefore organized into the following sections: Section 2 provides the requisite mathematical background on 3D rotation representations, including a naive attitude estimation using only gyroscope readings. Section 3 details the Madgwick filter, performing more accurate attitude estimation. Section 4 likewise discusses the Extended Kalman Filter. Section 5 covers the 6-DOF implementation of these estimators on a bare-metal microcontroller. Finally, Section 6 details any future work and concludes this writing.

2. Mathematical Background

Rotations can be represented through DCMs, Euler angles and quaternions. This project uses quaternions, justifying this approach from first-principles in this section.

2.1. Direct Cosine Matrices (DCMs)

Objects in 3D space, such as vectors or coordinate frames maintain a notion of orientation, which can be represented through a rotation matrix. Orientations and rotations are described with respect to a reference frame. A **reference frame** is a coordinate frame consisting of a set of orthonormal (perpendicular, unit length) basis vectors $\{\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3\}^T$. Any vector $\mathbf{v}$ in a coordinate frame can then be expressed as a linear combination of these basis vectors:

$$\mathbf{v} = a_1 \mathbf{e}_1 + a_2 \mathbf{e}_2 + a_3 \mathbf{e}_3 \quad (1)$$

The same object can be represented in with respect to different reference frames: for two coordinate frames $\mathcal{F}_A$ and $\mathcal{F}_B$, the same vector can be given by the following equivalence. Note that the scalar multiples $\mathbf{a}$ in frame A and $\mathbf{a}'$ in frame B would be different if the two coordinate frames are not aligned.

$$[\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3] \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = [\mathbf{e}'_1, \mathbf{e}'_2, \mathbf{e}'_3] \begin{bmatrix} a'_1 \\ a'_2 \\ a'_3 \end{bmatrix} \quad (2)$$

This equation can then be manipulated to determine the mapping of the vector coordinates in one frame to its equivalent coordinates in the other frame. This mapping is a 3x3 orthogonal matrix called the **rotation matrix $\mathbf{R}$** . It is constructed from the inner products of the basis vectors. These components are simply cosines, and therefore this matrix is also called a Direct Cosine Matrix (DCM):

$$\mathbf{R} = \begin{bmatrix} \mathbf{e}_1^T \mathbf{e}'_1 & \mathbf{e}_1^T \mathbf{e}'_2 & \mathbf{e}_1^T \mathbf{e}'_3 \\ \mathbf{e}_2^T \mathbf{e}'_1 & \mathbf{e}_2^T \mathbf{e}'_2 & \mathbf{e}_2^T \mathbf{e}'_3 \\ \mathbf{e}_3^T \mathbf{e}'_1 & \mathbf{e}_3^T \mathbf{e}'_2 & \mathbf{e}_3^T \mathbf{e}'_3 \end{bmatrix} \quad (3)$$

In order to prove this property of orthogonality, consider the definition of rotation. A rotation is a rigid-body transformation, such that the distance and

angle between any two points on an object are preserved after the transformation. Thus a rotated vector $\mathbf{v}$ should have the same magnitude as its original counterpart:

$$\|\mathbf{R}\mathbf{v}\|^2 = \|\mathbf{v}\|^2 \quad (4)$$

$$(\mathbf{R}\mathbf{v})^T(\mathbf{R}\mathbf{v}) = \mathbf{v}^T\mathbf{v} \quad (5)$$

$$\mathbf{v}^T\mathbf{R}^T\mathbf{R}\mathbf{v} = \mathbf{v}^T\mathbf{v} \quad (6)$$

This holds for all $\mathbf{v}$ only if $\mathbf{R}^T\mathbf{R} = \mathbf{I}$, the orthogonality condition.

In addition, the angle between two vectors $\mathbf{u}$ and $\mathbf{v}$ should be preserved after rotation. Consider the rotated vectors $\mathbf{R}\mathbf{u}$ and $\mathbf{R}\mathbf{v}$. Their dot product is:

$$(\mathbf{R}\mathbf{u}) \cdot (\mathbf{R}\mathbf{v}) = (\mathbf{R}\mathbf{u})^T(\mathbf{R}\mathbf{v}) \quad (7)$$

$$= \mathbf{u}^T\mathbf{R}^T\mathbf{R}\mathbf{v} \quad (8)$$

$$= \mathbf{u} \cdot \mathbf{v} \quad (9)$$

Thus, in addition to preserving distance (6), in order to preserve angles, $\mathbf{R}^T\mathbf{R} = \mathbf{I}$, again displaying the orthogonality condition.

Given its unit-length basis vectors, the determinant of $\mathbf{R}$ is either +1 or -1. A determinant of -1 represents a reflection of a coordinate frame, and is therefore not considered a valid rotation. Thus, a rotation matrix can be defined as any 3x3 orthogonal matrix with a determinant of +1. This group of all such matrices is labeled the **Special Orthogonal Group SO(3)**:

$$\text{SO}(3) = \{\mathbf{R} \in \mathbb{R}^{3 \times 3} \mid \mathbf{R}^T\mathbf{R} = \mathbf{I}, \det(\mathbf{R}) = +1\} \quad (10)$$

2.2. Order of Multiplication

The **SO(3)** group is closed under multiplication, so that rotation matrices can be chained together, yielding another rotation matrix. However, it is also important to note that the product of two rotation matrices is not always commutative (by the definition of a matrix group). Thus, the order of multiplication has significance.

Intuitively, from (2), by isolating the coordinate vector on the left side, the rotation matrix mapping the coordinate vectors a from frame B to A can be determined. The two matrices in the equation encode the orientation of their respective frames. Mathematically, their product is a rotation matrix that represents a coordinate transformation of frame B to A.

It is calculated as the right-multiplication of the basis vectors of frame B with those of frame A:

$$\mathbf{R}_B^A = \begin{bmatrix} \mathbf{e}_1 \\ \mathbf{e}_2 \\ \mathbf{e}_3 \end{bmatrix} [\mathbf{e}'_1, \mathbf{e}'_2, \mathbf{e}'_3] \quad (11)$$

When a rotation is made with respect to a non-global reference frame, the new rotation is said to be made intrinsically. When rotations are applied intrinsically, they compose through right-multiplication via **post-multiplication**. Post-multiplication of rotation matrices is applied when performing incremental coordinate transformations in a recursive fashion against the last updated orientation. Intrinsic rotations and post-multiplication lend themselves very well to IMU measurements made with respect to a given body-frame.

Pre-multiplication or left-multiplication of extrinsic rotations can also be performed. This operation can be used to change the original reference frame in the composition chain. In this project, only post-multiplication is used.

As a final point, it should be noted that a rotation matrix or DCM is computationally space-inefficient, requiring 9 floating point values. From Euler's Axis-Angle theorem, an equivalent rotation can be represented using just 4 parameters: a 3-tuple to describe an axis of rotation in 3D space, and a scalar to encode the magnitude of rotation about that axis.

2.3. Euler Angles & Gimbal-Lock

Euler Angles are not a major focus of this paper. They are mentioned here for their intuitive visualizations and to demonstrate gimbal-lock, making them unstable for use in a practical AHRS. Euler Angles are an alternate technique to represent rotations as the composition of three rotations about three discrete axes: the roll, pitch and yaw axes. Suppose by convention, a yaw-pitch-roll (ZYX) rotation sequence is applied, where:

$$R(\psi, \theta, \phi) = R_z(\psi) R_y(\theta) R_x(\phi) \quad (12)$$

and

$$R_z(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}, \quad R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix} \quad (13)$$

The rotation matrix will take on this general form:

$$R(\psi, \theta, \phi) = \begin{bmatrix} \cos \psi \cos \theta & \cos \psi \sin \theta \sin \phi - \sin \psi \cos \phi & \cos \psi \sin \theta \cos \phi + \sin \psi \sin \phi \\ \sin \psi \cos \theta & \sin \psi \sin \theta \sin \phi + \cos \psi \cos \phi & \sin \psi \sin \theta \cos \phi - \cos \psi \sin \phi \\ -\sin \theta & \cos \theta \sin \phi & \cos \theta \cos \phi \end{bmatrix} \quad (14)$$

Setting the pitch $\theta = \frac{\pi}{2}$, so that the yaw axis (world Z) and roll axis (body X) are aligned:

$$R\left(\psi, \frac{\pi}{2}, \phi\right) = \begin{bmatrix} 0 & \cos \psi \sin \phi - \sin \psi \cos \phi & \cos \psi \cos \phi + \sin \psi \sin \phi \\ 0 & \sin \psi \sin \phi + \cos \psi \cos \phi & \sin \psi \cos \phi - \cos \psi \sin \phi \\ -1 & 0 & 0 \end{bmatrix} \quad (15)$$

Applying trigonometric sum/difference identities, the rotation matrix then collapses to:

$$R\left(\psi, \frac{\pi}{2}, \phi\right) = \begin{bmatrix} 0 & \sin(\phi - \psi) & \cos(\phi - \psi) \\ 0 & \cos(\phi - \psi) & -\sin(\phi - \psi) \\ -1 & 0 & 0 \end{bmatrix} \quad (16)$$

The rotation matrix now depends only on the single parameter $(\phi - \psi)$, not independently on ϕ and ψ . Thus, it is no longer possible to independently adjust rotations about those aligned directions, resulting in the loss of a degree of freedom. This is called **gimbal lock**, and is an undesirable effect of chaining certain rotations. Euler angles are only used in this project as a human-readable interpretation of a rotation.

2.4. Quaternions

A quaternion $\mathbf{q} \in \mathbb{H}$ (the quaternion algebra) is a 4-tuple:

$$\mathbf{q} = (q_w, q_x, q_y, q_z) = q_w + q_x \mathbf{i} + q_y \mathbf{j} + q_z \mathbf{k} \quad (17)$$

where $q_w \in \mathbb{R}$ is the **scalar** component (real) and $\mathbf{v} = (q_x, q_y, q_z)^T \in \mathbb{R}^3$ is the **vector** component (imaginary). Equivalently:

$$\mathbf{q} = [s, \mathbf{v}]^T \quad (18)$$

where $s = q_w$ and $\mathbf{v} = [q_x, q_y, q_z]^T$.

Quaternion multiplication is governed by Hamilton's fundamental equation:

$$\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1 \quad (19)$$

From this equation, the following relationships can be derived:

$$\mathbf{ij} = \mathbf{k}, \quad \mathbf{ji} = -\mathbf{k} \quad (20)$$

$$\mathbf{jk} = \mathbf{i}, \quad \mathbf{kj} = -\mathbf{i} \quad (21)$$

$$\mathbf{ki} = \mathbf{j}, \quad \mathbf{ik} = -\mathbf{j} \quad (22)$$

Due to these relationships, quaternion multiplication is **non-commutative**. This can be proven through a simple distribution operation (omitted here for brevity). The Hamiltonian product of two quaternions is derived as:

For $\mathbf{q}_1 = (a, b, c, d)$ and $\mathbf{q}_2 = (e, f, g, h)$:

$$\mathbf{q}_1 \otimes \mathbf{q}_2 = \begin{bmatrix} ae - bf - cg - dh \\ af + be + ch - dg \\ ag - bh + ce + df \\ ah + bg - cf + de \end{bmatrix}$$

Note that quaternion multiplication is **associative**: $\mathbf{q}_1(\mathbf{q}_2\mathbf{q}_3) = (\mathbf{q}_1\mathbf{q}_2)\mathbf{q}_3$.

2.5. Quaternion Properties

In this section, some common quaternion properties are discussed. These are referenced repeatedly in Section 3. The following subsection also extends these general properties to describe a naive attitude estimation using only gyroscope readings.

2.5.1. Length (Norm):

The length of a quaternion is given by its L2 norm.

$$\|\mathbf{q}\| = \sqrt{q_w^2 + q_x^2 + q_y^2 + q_z^2} \quad (24)$$

The norm of a product of two quaternions equals the product of the norms of each quaternion. A unit quaternion has a norm of 1. Thus, the norm of a product of unit quaternions yields a unit quaternion.

$$\|\mathbf{q}_1 \otimes \mathbf{q}_2\| = \|\mathbf{q}_1\| \|\mathbf{q}_2\| \quad (25)$$

2.5.2. Conjugate and Inverse:

The conjugate negates the imaginary component (essentially reversing the axis of rotation), while maintaining the scalar component.

$$\mathbf{q}^* = (q_w, -q_x, -q_y, -q_z) = [s, -\mathbf{v}]^T \quad (26)$$

The inverse of a quaternion $\mathbf{q}$ is:

$$\mathbf{q}^{-1} = \frac{\mathbf{q}^*}{\|\mathbf{q}\|^2} \quad (27)$$

For unit quaternions ($\|\mathbf{q}\| = 1$), the inverse is therefore given by:

$$\mathbf{q}^{-1} = \mathbf{q}^* \quad (28)$$

Thus, inverting a rotation given by a unit quaternion can be done by:

$$\mathbf{q}^* \mathbf{q} = \mathbf{q} \mathbf{q}^* = 1 \quad (29)$$

2.5.3. Expressing Quaternions Using an Exponential Map

Using Euler's formula and applying an **exponential map** on the quaternion unit sphere, a unit quaternion can be represented below. More details on exponential mapping (including the half-angle property) can be found through a study of Lie Group theory[1]. Lie Group based attitude estimation is also currently applied, but falls outside the scope of this project. In scalar-vector form, $\mathbf{q} = [s, \mathbf{v}]^T$ where:

$$s = \cos \frac{\theta}{2} \quad (30)$$

$$\mathbf{v} = \sin \frac{\theta}{2} \hat{\mathbf{n}} \quad (31)$$

2.6. Rotations using Quaternions

A vector in 3D space $\mathbf{v}$ can be represented using a purely imaginary quaternion, where the scalar component is 0. In addition, for any 3D rotation that can be expressed using Axis-Angle, an equivalent unit quaternion exists, encoding this same rotation. Thus, in order to apply a rotation to a desired point, the quaternion "sandwich product formula" [2] can be applied:

First, the vector $\mathbf{v}$ is embedded as a pure imaginary quaternion:

$$\mathbf{p} = \begin{bmatrix} 0 \\ v_x \\ v_y \\ v_z \end{bmatrix} = [0, \mathbf{v}]^T \quad (32)$$

Next, the sandwich product formula is applied:

$$\mathbf{p}' = \mathbf{q} \otimes \mathbf{p} \otimes \mathbf{q}^* \quad (33)$$

Finally, the rotated vector can be extracted:

$$\mathbf{v}' = \begin{bmatrix} p'_x \\ p'_y \\ p'_z \end{bmatrix} \quad (\text{discard scalar component } p'_w = 0) \quad (34)$$

As an example, to perform three rotations on $\mathbf{v}$ in sequence, yielding the rotated vector $\mathbf{v}'$: unit rotation quaternions $\mathbf{p}$, then $\mathbf{q}$, then $\mathbf{r}$, are each applied with respect to the current orientation of the system (called the body-frame):

$$\text{first, apply } \mathbf{p}: \quad \mathbf{v}_1 = \mathbf{p} \mathbf{v} \mathbf{p}^* \quad (35)$$

$$\text{next, apply } \mathbf{q}: \quad \mathbf{v}_2 = \mathbf{q} \mathbf{v}_1 \mathbf{q}^* = \mathbf{q} (\mathbf{p} \mathbf{v} \mathbf{p}^*) \mathbf{q}^* \quad (36)$$

$$\text{finally, apply } \mathbf{r}: \quad \mathbf{v}' = \mathbf{r} \mathbf{v}_2 \mathbf{r}^* = \mathbf{r} (\mathbf{q} (\mathbf{p} \mathbf{v} \mathbf{p}^*) \mathbf{q}^*) \mathbf{r}^* \quad (37)$$

Condensing this using associativity:

$$\mathbf{v}' = \mathbf{r} \mathbf{q} \mathbf{p} \mathbf{v} \mathbf{p}^* \mathbf{q}^* \mathbf{r}^* \quad (38)$$

$$= (\mathbf{r} \otimes \mathbf{q} \otimes \mathbf{p}) \mathbf{v} (\mathbf{r} \otimes \mathbf{q} \otimes \mathbf{p})^* \quad (39)$$

Thus, the composed rotation quaternion is:

$$\mathbf{q}_{\text{total}} = \mathbf{r} \otimes \mathbf{q} \otimes \mathbf{p} \quad (40)$$

and its conjugate can be derived by:

$$(\mathbf{r} \otimes \mathbf{q} \otimes \mathbf{p})^* = \mathbf{p}^* \otimes \mathbf{q}^* \otimes \mathbf{r}^* \quad (41)$$

Note: This is intrinsic (body-frame) composition via post-multiplication, consistent with the reasoning from Section 2.2. The transformed vector can therefore be written as:

$$\mathbf{v}' = \mathbf{q}_{\text{total}} \mathbf{v} \mathbf{q}_{\text{total}}^* \quad (42)$$

2.7. Gyroscope Updates & Quaternion Time Derivative

The attitude estimate is recursively updated through a standard first-order Euler integration, such that the prior attitude estimate is updated based on the current rate of change of attitude, over a small, discrete increment of time:

$$\mathbf{q}_{k+1} = \mathbf{q}_k + \dot{\mathbf{q}}_k \Delta t \quad (43)$$

The onboard IMU measures angular velocities $[\frac{rad}{sec}]$ where $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^T$ measured with respect to the body frame. The body frame is the coordinate frame that is directly attached to the sensor or its containing body. An imaginary quaternion encoding these velocities is therefore post-multiplied with the current attitude through the Quaternion Derivative Formula[2]:

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes [0, \boldsymbol{\omega}^T]^T = \frac{1}{2} \mathbf{q} \otimes \boldsymbol{\omega}_q \quad (44)$$

To propagate orientation, we integrate the quaternion time derivative:

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \boldsymbol{\omega}_q = \frac{1}{2} \mathbf{q} \otimes \begin{bmatrix} 0 \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \quad (45)$$

where $\boldsymbol{\omega}_q = [0, \boldsymbol{\omega}^T]^T$ is the angular velocity embedded as a pure quaternion.

2.7.1. Aside: Skew-Symmetric Equivalence

The quaternion multiplication $\mathbf{q} \otimes \boldsymbol{\omega}_q$ can be expressed as a linear matrix operation. From the Hamiltonian product:

$$\mathbf{q} \otimes \boldsymbol{\omega}_q = \begin{bmatrix} q_w \cdot 0 - q_x \omega_x - q_y \omega_y - q_z \omega_z \\ q_w \omega_x + q_x \cdot 0 + q_y \omega_z - q_z \omega_y \\ q_w \omega_y - q_x \omega_z + q_y \cdot 0 + q_z \omega_x \\ q_w \omega_z + q_x \omega_y - q_y \omega_x + q_z \cdot 0 \end{bmatrix} \quad (46)$$

$$= \begin{bmatrix} -q_x \omega_x - q_y \omega_y - q_z \omega_z \\ q_w \omega_x + q_y \omega_z - q_z \omega_y \\ q_w \omega_y - q_x \omega_z + q_z \omega_x \\ q_w \omega_z + q_x \omega_y - q_y \omega_x \end{bmatrix} \quad (47)$$

Rewriting this expression in matrix-vector multiplication form, a skew-symmetric matrix is derived. In this writing, the skew-symmetric of a given vector $\boldsymbol{\omega}$ might sometimes be also be written as $[\boldsymbol{\omega}_\times]$. Thus, $\boldsymbol{\Omega}(\boldsymbol{\omega})$ here is equivalent to $[\boldsymbol{\omega}_\times]$.

$$\mathbf{q} \otimes \boldsymbol{\omega}_q = \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix} \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix} = \boldsymbol{\Omega}(\boldsymbol{\omega}) \mathbf{q} \quad (48)$$

In addition, note that the transpose of a skew-symmetric matrix is its negative counterpart, so that $\boldsymbol{\Omega}^T = -\boldsymbol{\Omega}$.

In this manner, the quaternion time derivative equation can be written as a simple matrix-vector multiplication, which is easily parallelized using SIMD or other vectorization architectures, improving filter computation frequency. This also significantly simplifies the linearization step in Extended Kalman Filter, discussed later. Finally, this form fits neatly into the classic state-space framework used in control theory.

It should be noted that simple Euler integration approach provides an uncorrected orientation estimate based solely on **raw** gyroscope data. **Given that gyroscope is inherently prone to drift and bias errors, the integration begins to amplify this error, quickly rendering this attitude estimate useless.**

3. Attitude Estimation: Madgwick Filter

The Madgwick filter is a computationally efficient orientation estimator that fuses 6-DOF IMU data (gyroscope and accelerometer) using quaternion representation. It combines gyroscope integration with gradient descent correction

based on accelerometer measurements. The filter performs in two steps every timestep: a prediction step during which gyroscope angular velocity is integrated to propagate orientation (as seen in Section 2.7). Next, a correction step is applied, using gradient descent to minimize error between measured and predicted gravity direction.

3.1. Prediction Step

The prediction step at timestep t is as derived in (45). Below, the term $\hat{\mathbf{q}}_t^-$ is called the **prior** attitude estimate (prior is denoted by the superscript $-$ and the hat symbol denotes that this is an estimate). This is the uncorrected estimate, made purely using the final (or **posterior**, denoted by the superscript $+$) estimate from the previous timestep $t - 1$ and the current gyroscope sensor reading:

$$\dot{\mathbf{q}}_t = \frac{1}{2} \hat{\mathbf{q}}_{t-1}^+ \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega}_t \end{bmatrix} \quad (49)$$

$$\hat{\mathbf{q}}_t^- = \hat{\mathbf{q}}_{t-1}^+ + \dot{\mathbf{q}}_t \Delta t \quad (50)$$

Any gyroscope bias or measurement noise accumulates unbounded over time, causing **orientation drift**. After minutes of integration, the estimated orientation can deviate significantly from the true orientation.

3.2. Accelerometer as Absolute Reference

An accelerometer provides an **absolute measurement** of the gravity direction in the body frame. Unlike gyroscopes, accelerometer readings do not accumulate error over time when a stationary assumption is made. Therefore, by comparing the gyroscope-predicted gravity vector with the accelerometer-estimated gravity vector, drift error can be estimated and corrected recursively.

3.3. Stationary Condition

The stationary condition is defined to assume zero **linear acceleration**, not angular velocity. Thus, as the IMU rotates, the accelerometer only measures gravity in the body frame. The centripetal acceleration is also typically negligible although the accelerometer reading could be affected by higher angular velocities, as given by:

$$\mathbf{a}_{\text{centripetal}} = \omega^2 r \quad (51)$$

By physically centering the IMU with the axis of rotation ($r = 0$), this centripetal acceleration can be negated. However, this pinpoints an inherent

issue with the Madgwick Filter: at higher angular rates, the contribution of the accelerometer cannot be dynamically reduced to trust the gyroscope reading more.

3.4. Correction Step

In order to determine how much to adjust the prior attitude estimate, an optimization problem is defined, where the objective function represents the error between the predicted gravity vector versus the estimated gravity vector measured by the accelerometer, in the body frame.

3.4.1. Objective Function:

The predicted gravity vector quaternion based on the gyroscope reading can be determined simply using the gravity vector in the world frame, represented here with only a $\mathbf{q}_z$ component. It is first put in terms of the body frame. The quaternion sandwich product (33) can be applied to perform this transformation:

$$\hat{\mathbf{g}}_{\text{pred}}^b(\hat{\mathbf{q}}_t^-) = (\hat{\mathbf{q}}_t^-)^* \otimes \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \otimes \hat{\mathbf{q}}_t^- \quad (52)$$

Applying the Hamiltonian product, this expression simplifies to [4]:

$$\hat{\mathbf{g}}_{\text{pred}}^b(\hat{\mathbf{q}}_t^-) = \begin{bmatrix} 0 \\ 2(q_x q_z - q_w q_y) \\ 2(q_w q_x + q_y q_z) \\ 1 - 2q_x^2 - 2q_y^2 \end{bmatrix} \quad (53)$$

Next, let $\hat{\mathbf{a}}$ be the estimated gravity vector quaternion, measured by the accelerometer. The error between these two vectors represents the objective function $\mathbf{f}_g(\hat{\mathbf{q}}, \hat{\mathbf{a}})$:

$$\mathbf{f}_g(\hat{\mathbf{q}}_t^-, \hat{\mathbf{a}}) = \hat{\mathbf{g}}_{\text{pred}}^b(\hat{\mathbf{q}}_t^-) - \hat{\mathbf{a}} \quad (54)$$

3.4.2. Gradient Descent:

A gradient descent operation can now be applied using this objective function. Through the standard chain rule, the gradient of the error function $\mathbf{f}_g$ with respect to the predicted attitude estimate is:

$$\nabla \mathbf{f}_g = \mathbf{J}_f^T \cdot \mathbf{f}_g(\hat{\mathbf{q}}, \hat{\mathbf{a}}) \quad (55)$$

where $\mathbf{J}_f$ is the Jacobian matrix of the error function[CITATION, MADGWICK, (26)]:

$$\mathbf{J}_f = \frac{\partial \mathbf{f}_g}{\partial \hat{\mathbf{q}}_t} = \begin{bmatrix} -2q_y & 2q_z & -2q_w & 2q_x \\ 2q_x & 2q_w & 2q_z & 2q_y \\ 0 & -4q_x & -4q_y & 0 \end{bmatrix} \quad (56)$$

The gradient can then be normalized and negated, representing the direction of steepest descent of error in quaternion space. A filter gain β is used to control rate of convergence. Intuitively, this parameter encodes "trust" in the accelerometer estimate.

$$\dot{\mathbf{q}}_{\nabla} = -\beta \frac{\nabla_{\hat{\mathbf{q}}} \mathbf{f}_g}{\|\nabla_{\hat{\mathbf{q}}} \mathbf{f}_g\|} \quad (57)$$

The filter gain β represents the magnitude of gyroscope measurement error and is defined by [4]:

$$\beta = \sqrt{\frac{3}{4}} \tilde{\omega}_{\beta} \quad (58)$$

where $\tilde{\omega}_{\beta}$ is a fixed parameter encoding the estimated mean zero gyroscope measurement error in radians per second, typically obtained from the sensor datasheet. Thus, the output of (57) is a quaternion derivative.

The estimated rate of change of attitude quaternion can thereafter be corrected using this value (57). This is called fusing:

$$\dot{\mathbf{q}}_t = \dot{\mathbf{q}}_t + \dot{\mathbf{q}}_{\nabla} \quad (59)$$

Finally, the posterior attitude estimate can be derived through numerical integration and then normalized to maintain a magnitude of 1:

$$\hat{\mathbf{q}}_t^+ = \hat{\mathbf{q}}_{t-1}^+ + \dot{\mathbf{q}}_t \Delta t \quad (60)$$

$$\hat{\mathbf{q}}_t^+ = \frac{\hat{\mathbf{q}}_t^+}{\|\hat{\mathbf{q}}_t^+\|} \quad (61)$$

4. Attitude Estimation Using Quaternions: EKF

The Kalman Filter recursively estimates the state of a linear system by fusing predictions based on a motion model with measurements from sensors, weighing each contribution based on their respective uncertainties. It thus maintains a state estimate and uncertainty (covariance), updated with each new measurement. The Kalman Filter expects a linear system since this covariance is modeled as a Gaussian distribution. Thus, a linear system transforms a Gaussian input into a Gaussian output, ideal for propagating confidence in a given attitude estimate

over time.

The Extended Kalman Filter (EKF) can be used to apply a Kalman Filter scheme to a non-linear system (such as to model quaternion rotations). The EKF performs a linear approximation of the motion and measurement models, thereafter applying a standard Kalman Filter technique on the system. In this section, these steps are given in more detail, while also optionally integrating magnetometer readings to track yaw.

This paper specifically focuses on the Error State Kalman Filter (ESKF) [1], a flavor of EKF that corrects around an error state. Sola describes the ESKF as follows:

"High-frequency IMU data is integrated into a nominal-state $\mathbf{x}$. This nominal state does not take into account the noise terms and other possible model imperfections. As a consequence, it will accumulate errors. These errors are collected in the error-state $\mathbf{x}$ and estimated with the Error-State Kalman Filter (ESKF), this time incorporating all the noise and perturbations. The error-state consists of small-signal magnitudes, and its evolution function is correctly defined by a (time-variant) linear dynamic system, with its dynamic, control and measurement matrices computed from the values of the nominal-state." [1]

4.1. Prediction Step

The prediction step propagates the posterior attitude estimate and its associated error covariance from the previous timestamp, using a new gyroscope measurement ω . Similarly to the Madgwick Filter prediction step (Section 3.1), the prior estimate can be calculated by:

$$\dot{\mathbf{q}}_t = \frac{1}{2} \hat{\mathbf{q}}_{t-1}^+ \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega}_t \end{bmatrix} \quad (62)$$

$$\boxed{\hat{\mathbf{q}}_t^- = \hat{\mathbf{q}}_{t-1}^+ + \dot{\mathbf{q}}_t \Delta t} \quad (63)$$

where $\mathbf{q}_{t-1}^+ = [w, x, y, z]^T$ is the unit quaternion representing the posterior attitude estimate and $\boldsymbol{\omega}_t = [0, \omega_x, \omega_y, \omega_z]^T$ is the pure quaternion representation of the angular velocity vector $\boldsymbol{\omega}$ currently measured by the gyroscope in rad/s.

An error dynamics $\delta\dot{\boldsymbol{\theta}}$ encoding the change in nominal state versus true state (true attitude is an unknowable quantity) can be derived on this basis, with its linearized form shown below [1], in continuous-domain, linear state-space form:

$$\delta\dot{\boldsymbol{\theta}} = -[\boldsymbol{\omega}_\times] \delta\boldsymbol{\theta} + \mathbf{w}_{gyro} \quad (64)$$

where $\delta\boldsymbol{\theta} \in \mathbb{R}^3$ is the rotation error vector, $[\boldsymbol{\omega}_\times]$ is the skew-symmetric of the measured angular velocity, and $\mathbf{w}_{gyro} \sim \mathcal{N}(0, Q)$ is gyroscopic sensor white-noise.

The skew-symmetric $(-\omega_{\times})$ represents the state transition matrix F , encoding the attitude dynamics.

The discrete-time state transition matrix Φ is derived by using the error dynamics to formulate the following single timestep integration:

$$\dot{\delta\theta}(t) = F \delta\theta(t) \quad (65)$$

$$\implies \delta\theta(t + \Delta t) = \delta\theta(t) + \int_t^{t+\Delta t} F \delta\theta(\tau) d\tau \quad (66)$$

A first-order Euler approximation is applied, simplifying (69):

$$\delta\theta(t + \Delta t) \approx \delta\theta(t) + F \delta\theta(t) \Delta t \quad (67)$$

$$\implies \delta\theta_{k+1} \approx (I + F\Delta t) \delta\theta_k \quad (68)$$

$$\implies \Phi = I_3 - [\omega_{\times}] \Delta t \quad (69)$$

The discrete-time prior covariance matrix can then derived:

$$\boxed{P_{k+1}^- = \Phi P_k^+ \Phi^T + Q \Delta t} \quad (70)$$

4.2. Correction Step:

Once again, the correction step will be driven by the error between the accelerometer-estimated gravity vector and the predicted gravity vector (both in body frame) as a result of the prior estimate. This quantity is called the innovation vector $\mathbf{y}$. This is the same expression used in the Madgwick Filter (Eq. 52):

$$\mathbf{y} = \mathbf{f}_g(\hat{\mathbf{q}}_t^-, \hat{\mathbf{a}}) = \hat{\mathbf{g}}_{\text{pred}}^b - \hat{\mathbf{a}} \quad (71)$$

Similarly to the prediction step, a linearization step must be applied to derive the posterior covariance matrix. For small rotation errors $\delta\theta$, a first order approximation of the perturbed gravity vector, through a cross-product is shown here:

$$\mathbf{g}_{\text{perturb}}^b \approx \mathbf{g}_{\text{pred}}^b + (\delta\theta \times \mathbf{g}_{\text{pred}}^b) \quad (72)$$

Using [2] and Eq. 48, the cross-product identity can be applied (notice the reversed order) to derive a matrix-vector multiplication such that:

$$\delta\theta \times \mathbf{g}_{pred}^b = -[\mathbf{g}_{pred}^b \times] \delta\theta \quad (73)$$

$$\mathbf{H} = -[\mathbf{g}_{pred}^b \times] \quad (74)$$

This forms a linearized measurement Jacobian $\mathbf{H}$, which maps an increment of state error to a corresponding change in the predicted gravity vector. Here, state error represents a measure of rotation (i.e. how much the predicted vector must be rotated to match the estimated vector).

Next, the innovation covariance S matrix is calculated, where R is additive measurement (accelerometer) sensor noise. This encodes the innovation uncertainty. The Kalman Gain K then defines a proportional scaling factor in response to the gravity error, used to determine the applied correctional rotation to the prior estimate:

$$S = H P_{k+1}^- H^T + R \quad (75)$$

$$K = P_{k+1}^- H^T S^{-1} \quad (76)$$

$$\delta\theta = K \mathbf{y} \quad (77)$$

Finally, the posterior state and covariance can be derived. The posterior attitude estimate is left-multiplied by the derived rotation increment in quaternion form. This correction quaternion represents the accumulated orientation error resolved in the global reference frame.

$$\boxed{\mathbf{q}_{k+1}^+ = \delta\mathbf{q}(\delta\theta) \otimes \mathbf{q}_{k+1}^-} \quad (78)$$

$$\boxed{P_{k+1}^+ = (I - KH)P_{k+1}^-(I - KH)^T + K R K^T} \quad (79)$$

5. Practical 6-DOF Implementation: Implementation & Findings

The system follows a layered architecture from hardware abstraction to application logic. At the lowest level, the Microcontroller Abstraction Layer (MCAL) provides STM32F446-specific peripheral configuration, including I²C bus initialization, clock tree setup, and GPIO configuration. The Driver Layer sits above this, implementing an MPU6050/MPU9250 IMU driver that abstracts register-level communication into high-level sensor read operations. The Math Utilities Layer provides quaternion algebra and wraps the Eigen linear algebra primitives for efficient matrix operations. At the top, the Filter Layer implements both EKF and Madgwick filters behind a common IAttitudeEstimator interface,

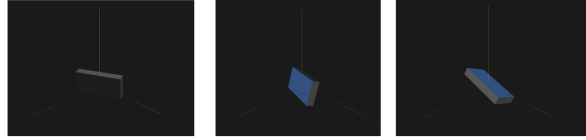


Figure 1: A demonstration of 6DOF estimation: from left-to-right, roll and pitch behavior, using the Madgwick and the EKF.

allowing runtime algorithm selection. The main application loop reads IMU data and retrieves orientation quaternions for downstream control algorithms. The IMU communicates over I²C (Inter-Integrated Circuit) protocol operating at 400 kHz in Fast Mode. The two-wire interface (SCL for clock, SDA for bidirectional data) uses open-drain signaling with external pull-up resistors.

Each sensor read follows a burst-read pattern: the CPU writes the starting register address, then the IMU streams 14 consecutive bytes containing accelerometer and gyroscope readings as 16-bit signed integers. At 400 kHz, this 14-byte transaction should complete in approximately 1-2 milliseconds. The current implementation uses CPU-driven polling, though DMA-based transfers could reduce interrupt latency for higher-rate applications.

Currently, the UART bus logging IMU readings on screen for visualization performs CPU blocking calls and therefore reduces the frequency of the filter significantly. By leveraging DMA in future to perform IMU-to-buffer and buffer-to-UART data register transfers, the filter would be significantly faster.

6. Results & Future Work

This project aims to provide a a first-pass implementation of these filters and visualize them to better understand their performance. The visualizations show that the I²C bus is correctly configured and the CPU is capable of running these filters on the bare-metal. With added RTOS functionality, multiplexing many filters concurrently for visualization is also an option.

From the results (as shown in the accompanying videos), more gain tuning and more robust calibration must be performed. Both filters suffer from gyroscope bias. The current calibration step is a simplification of more advanced techniques, simply treating bias as a constant offset to be negated each timestep.

7. Conclusion

In conclusion, this project serves a first step to implementing industry-standard attitude estimators from scratch. This serves as a first pass to map

the abstract mathematical concepts of sensor fusion to practical implementation on the bare-metal with no external library support. There is plenty of future work to improve this implementation: in a 6DOF configuration, leveraging non-blocking UART and using DMA to support telemetry transfer to peripherals would significantly improve filter frequency. In addition, more sophisticated calibration must be used to account for gyroscope drift error, beyond relying on the correction steps. However, gain tuning might mitigate this issue as well.

References

- [1] J. Solà, “Quaternion kinematics for the error-state Kalman filter,” arXiv preprint arXiv:1711.02508, 2017. [Online]. Available: <https://arxiv.org/abs/1711.02508>
- [2] X. Gao and T. Zhang, Introduction to Visual SLAM: From Theory to Practice. Beijing, China: Tsinghua University Press, 2021
- [3] M. García, AHRS: Attitude and Heading Reference Systems—Open Source Python Toolbox for Attitude Estimation, 2019–2025. [Online]. Available: <https://github.com/Mayitzin/ahrs>
- [4] S. O. H. Madgwick, An Efficient Orientation Filter for Inertial and Inertial/Magnetic Sensor Arrays, Internal Report, University of Bristol, U.K., Apr. 2010.